

```
import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
from tensorflow.keras.preprocessing.sequence import pad_sequences
import matplotlib.pyplot as plt
```

```
vocab_size = 10000
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=vocab_size)
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz>
17464789/17464789 ————— **0s** 0us/step

```
max_length = 200
X_train = pad_sequences(X_train, maxlen=max_length, padding='post')
X_test = pad_sequences(X_test, maxlen=max_length, padding='post')
```

```
model = Sequential()
model.add(Embedding(input_dim=vocab_size, output_dim=32, input_length=max_length))
model.add(LSTM(units=32))
model.add(Dense(units=1, activation='sigmoid'))
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument 'input_length' is deprecated. Use 'input_shape' instead.
 warnings.warn(

```
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

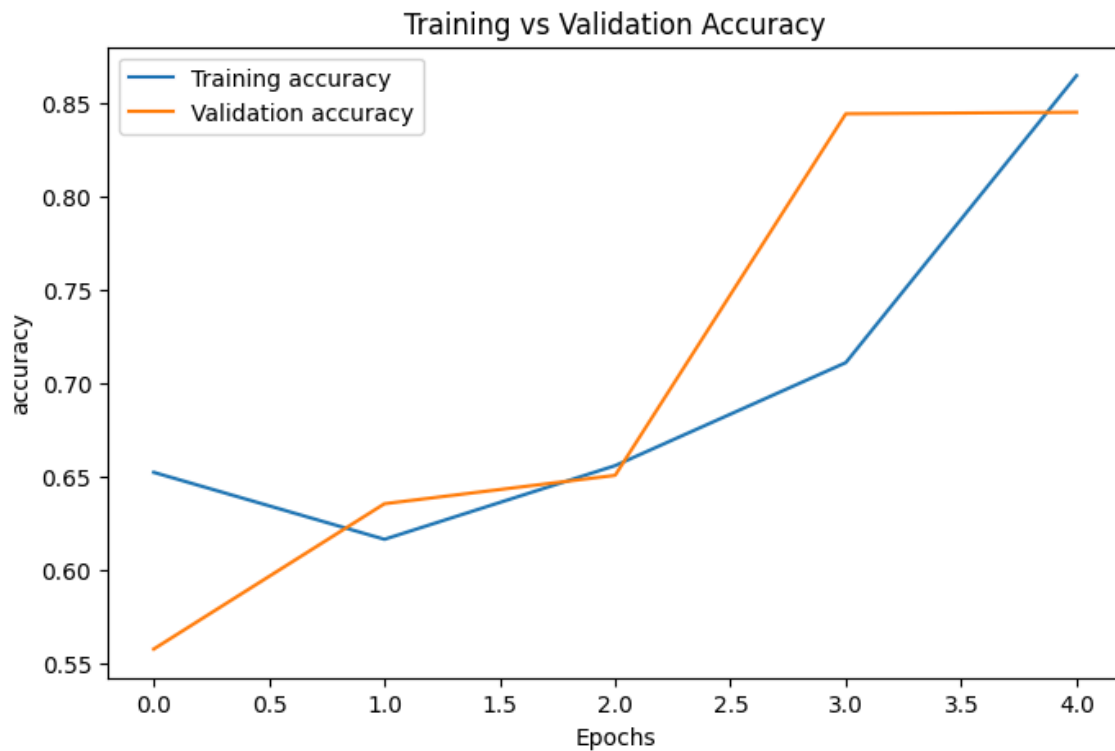
```
history = model.fit(X_train, y_train, epochs=5, batch_size=64, validation_split=0.2)
```

```
Epoch 1/5
313/313 ————— 18s 58ms/step - accuracy: 0.6524 - loss: 0.6209 - val_accuracy: 0.557
Epoch 2/5
313/313 ————— 18s 56ms/step - accuracy: 0.6166 - loss: 0.6383 - val_accuracy: 0.635
Epoch 3/5
313/313 ————— 18s 58ms/step - accuracy: 0.6560 - loss: 0.5537 - val_accuracy: 0.656
Epoch 4/5
313/313 ————— 20s 57ms/step - accuracy: 0.7111 - loss: 0.4954 - val_accuracy: 0.844
Epoch 5/5
313/313 ————— 19s 52ms/step - accuracy: 0.8648 - loss: 0.3614 - val_accuracy: 0.845
```

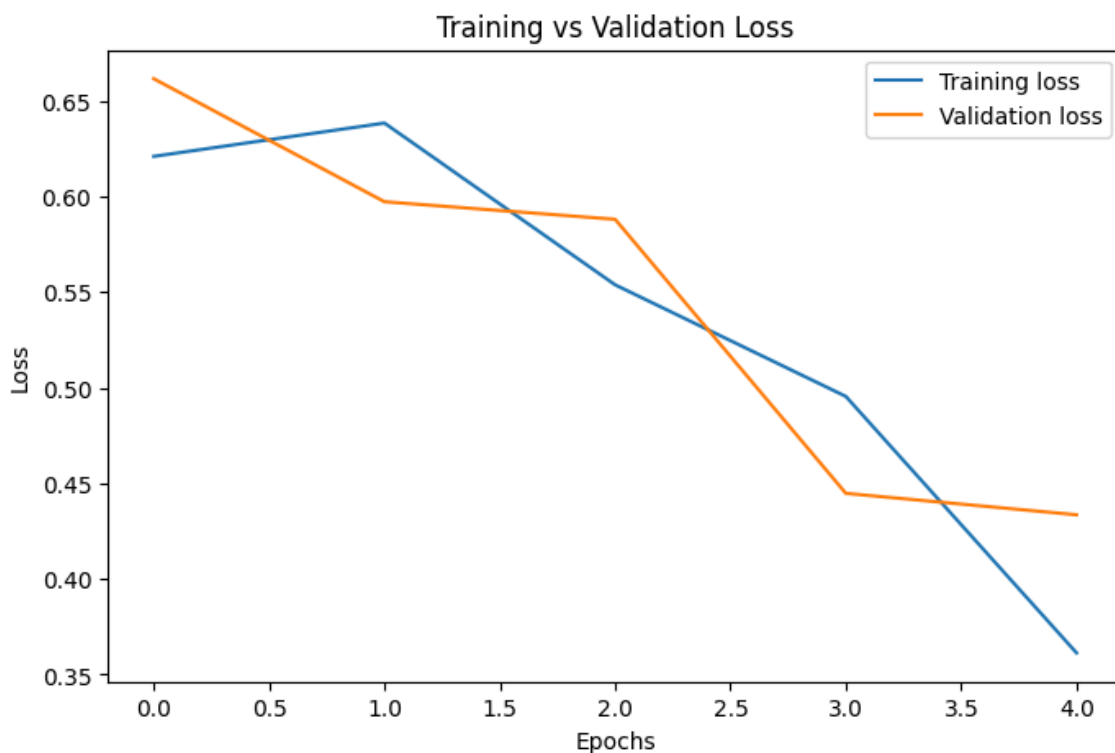
```
loss, accuracy = model.evaluate(X_test, y_test)
print("Test Loss:", loss)
print("Test Accuracy:", accuracy)
```

782/782 ————— **9s** 11ms/step - accuracy: 0.8390 - loss: 0.4433
 Test Loss: 0.4433176517486572
 Test Accuracy: 0.8389599919319153

```
plt.figure(figsize=(8, 5))
plt.plot(history.history['accuracy'], label='Training accuracy')
plt.plot(history.history['val_accuracy'], label='Validation accuracy')
plt.xlabel('Epochs')
plt.ylabel('accuracy')
plt.title("Training vs Validation Accuracy")
plt.legend()
plt.show()
```



```
plt.figure(figsize=(8, 5))
plt.plot(history.history['loss'], label='Training loss')
plt.plot(history.history['val_loss'], label='Validation loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title("Training vs Validation Loss")
plt.legend()
plt.show()
```



```
predictions = model.predict(X_test[:10])
for i, prediction in enumerate(predictions):
    print(f"Review {i + 1}")
    print("Predicted Probability:", prediction[0])
    if prediction >= 0.5:
```

```
        print("Predicted Sentiment: Positive")
    else:
        print("Predicted Sentiment: Negative")
    print("Actual Label:", y_test[i])
    print("-"*40)
```

1/1  0s 130ms/step

Review 1

Predicted Probability: 0.23114024

Predicted Sentiment: Negative

Actual Label: 0

Review 2

Predicted Probability: 0.9863108

Predicted Sentiment: Positive

Actual Label: 1

Review 3

Predicted Probability: 0.93211883

Predicted Sentiment: Positive

Actual Label: 1

Review 4

Predicted Probability: 0.23122111

Predicted Sentiment: Negative

Actual Label: 0

Review 5

Predicted Probability: 0.82375544

Predicted Sentiment: Positive

Actual Label: 1

Review 6

Predicted Probability: 0.23114036

Predicted Sentiment: Negative

Actual Label: 1

Review 7

Predicted Probability: 0.92770624

Predicted Sentiment: Positive

Actual Label: 1

Review 8

Predicted Probability: 0.22896773

Predicted Sentiment: Negative

Actual Label: 0

Review 9

Predicted Probability: 0.82350147

Predicted Sentiment: Positive

Actual Label: 0

Review 10

Predicted Probability: 0.9745127

Predicted Sentiment: Positive

Actual Label: 1
